

ELI V2.0 · SECURITY & PRIVACY

Security & Privacy

ELI is built to be private by construction, not by policy. Your data lives on your machine, network access is refused at the socket unless you allow it, and every privileged action is gated and recorded. This is exactly how that works — and how to verify it yourself.

Edition: v2.0.16 · **Audience:** buyers, administrators, and the security-minded · **Author:** ShadowESC95
Design goal: a local assistant you can hand real access to without handing your data to anyone else.

The threat model in one paragraph

ELI runs a language model on your own hardware and drives your computer. The risks that matter are therefore: data leaving your machine, the model reaching the internet without your say-so, and a privileged action (shell, file, install) happening without your knowledge. ELI is engineered against all three.

1 • Offline by default — enforced at the socket

ELI does all inference and memory locally. Internet access is opt-in via the **Net** toggle, and — critically — that isn't a courtesy check the code might forget to make. A **process-wide socket guard** (`eli/core/netguard.py`) is installed at startup and wraps the operating system's own connect primitives:

- `socket.socket.connect` and `connect_ex` — the synchronous paths (and the async event loop's connector).
- `socket.create_connection` — used by `urllib` / `requests` / `http.client`.
- The Windows Proactor event loop's connector, best-effort, for overlapped I/O.

While the Net toggle is off, any *non-loopback* connection is refused before a packet leaves — code that forgot to check the policy still fails closed at the socket boundary. Loopback and Unix sockets are always allowed (that's just the app talking to itself). Every *allowed* outbound connection is recorded, so the Net tab shows exactly what ELI reached and when.

Verify it yourself

Turn the Net toggle off and ask ELI to search the web. It refuses honestly rather than guessing. Turn it on, ask again, and watch the outbound host appear in the Net tab's egress log. Nothing is hidden.

2 • Full Control — the deliberate override

Power users can enable **Full Control**, which lifts the offline-by-default barrier. This is an explicit, opt-in setting — not a default, and not something ELI can turn on for itself. If web actions ever run while the Net toggle appears off, Full Control is the reason, and it is by design. Leave it off for the strictest posture.

3 • Every networked task routes through one gate

Beyond the socket failsafe, the features that *can* use the internet — web search, news, weather — each pass a single policy gate that fails closed when offline. Web search deserves a special note: its backend can drive libcurl at the C level, which bypasses Python's socket layer entirely, so the gate is enforced *inside the plugin* as well. Defense in depth: the socket guard and the plugin gate both have to be satisfied.

4 • Privileged actions are gated and confirmed

- **Shell & generated scripts.** Running a command or a generated script passes a risk gate and an approval step, and generated scripts run through a safety wrapper. Nothing executes silently on your behalf.
- **Installs & fixes.** When ELI wants to install an app or apply a code fix, it asks first and waits for your yes/no.
- **Habits.** Behaviours ELI notices are proposed *disabled*; it acts on a habit only after you approve it.

5 • The web server is access-controlled

When you expose ELI to your phone (LAN mode), the server is not open:

- **Token required.** LAN mode auto-generates a stable access token (persisted locally, 0600). The token rides in the URL *fragment*, so it never lands in server request logs. Rotate it from the Connect tab to kick paired devices off.
- **Role-based access.** The API distinguishes read-only viewers from members; acting endpoints (chat, execute) are refused (403) to viewers.
- **HTTPS for the microphone.** The phone mic requires HTTPS; ELI generates a self-signed LAN certificate locally (via `cryptography`) so the browser will grant microphone access — no external CA, no cloud.
- **Loopback stays tokenless.** On 127.0.0.1 (this machine only) there's no token friction; the gate fails closed the moment you bind beyond loopback.

6 • A tamper-evident audit trail

Every API action is written into a **hash-chained ledger** — who did what, with what outcome. Because each entry chains to the previous one's hash, silent edits to the history are detectable. The Audit surface exposes recent events, read-only.

7 • Where your data lives (and doesn't)

- **On your disk, in the install folder:** four SQLite databases (conversations, memories, your profile, agent + coding memory), a FAISS vector index, and your settings. Move or back up the folder and it's all there.
- **Never sent anywhere for inference.** Prompts, memories and files are processed by the local model. There is no telemetry phone-home and no cloud account.
- **No secrets in the shipped package.** The distributable contains empty template databases and example settings only — none of your runtime data. (Independently verified: the release tarball carries zero conversations, memories, tokens, logs, or machine paths.)

8 • Reporting a vulnerability

Found a security issue? Please report it privately to jaybridgeman0095@gmail.com rather than opening a public issue, and allow reasonable time for a fix before disclosure. See [SECURITY.md](#) in the package for the current policy.

ELI v2.0 — private by construction. Offline-by-default, socket-enforced, gated, audited. Your hardware, your data, your control.